

Quiz autoevaluación: Semana 3 (Scheduling)

Abrió: jueves, 5 de marzo de 2026, 08:00
Cierra: jueves, 12 de marzo de 2026, 23:59

Tema de la clase anterior

Quiz de autoevaluación: Semana 3 (MLFQ)

Abre: martes, 10 de marzo de 2026, 08:30
Cierra: martes, 17 de marzo de 2026, 23:59

Tema de esta clase

Pendientes

Ojala antes del sabado

Foro { User RH
Correo institucional

Apuntes clase 6

Sobre los cursos Reh Hat

Votación finalizada | 2 preguntas | 16 de 16 (100%) participaron

1. Ya creo su cuenta de Red Hat (Opción única) *

16/16 (100%) han respondido

Si	(8/16) 50%
No	(8/16) 50%

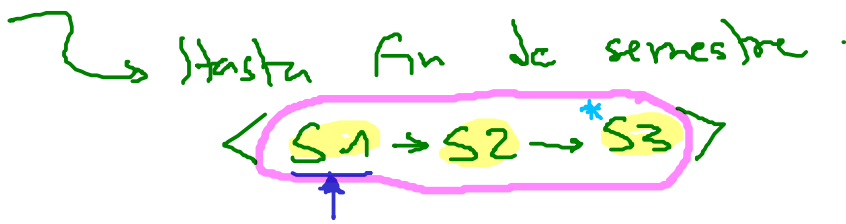
2. ¿Que inconvenientes ha tenido? (Opción única) *

16/16 (100%) han respondido

Ninguno, con la informacion del foro fue suficiente	(6/16) 38%
Nada, no he revisado el foro	(3/16) 19%
Ya revise el foro pero aun no lo he hecho	(7/16) 44%
No entendi el procedimiento del foro, la informacion esta confusa.	(0/16) 0%

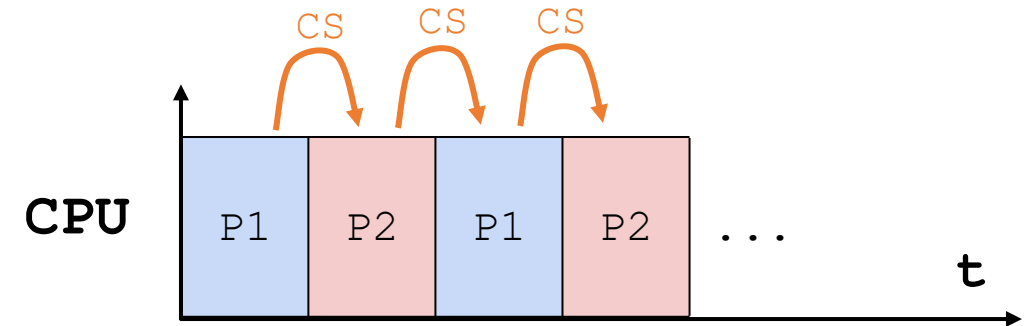
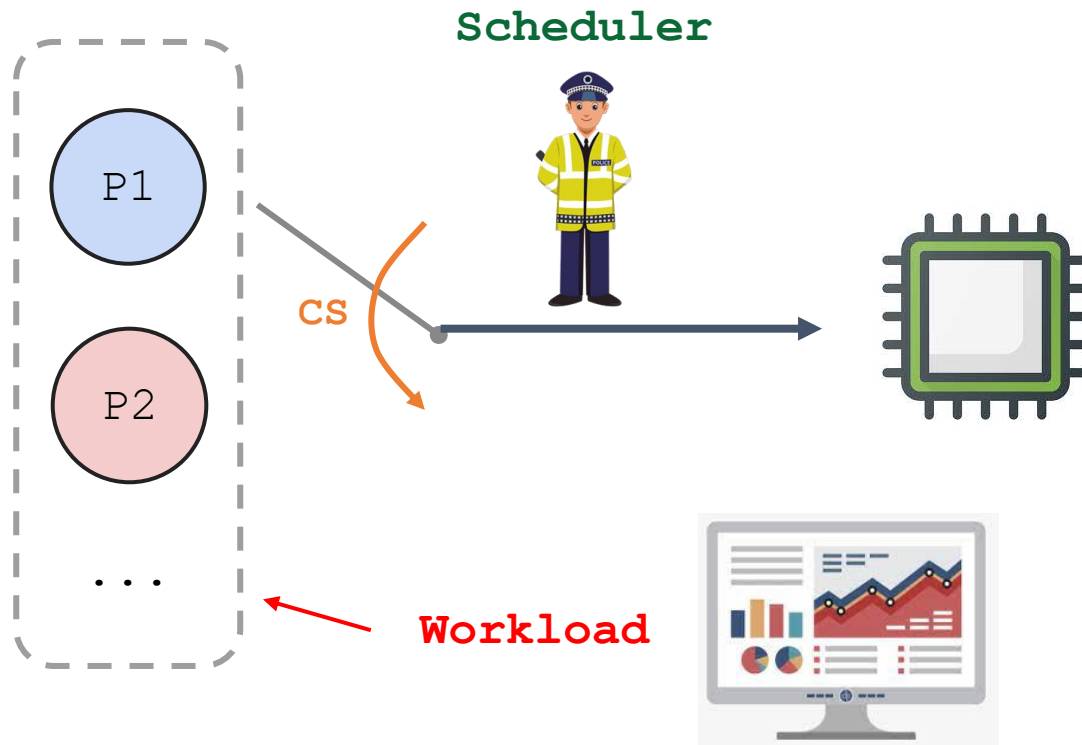
Multi-Level Feedback Queue

05/03/2026



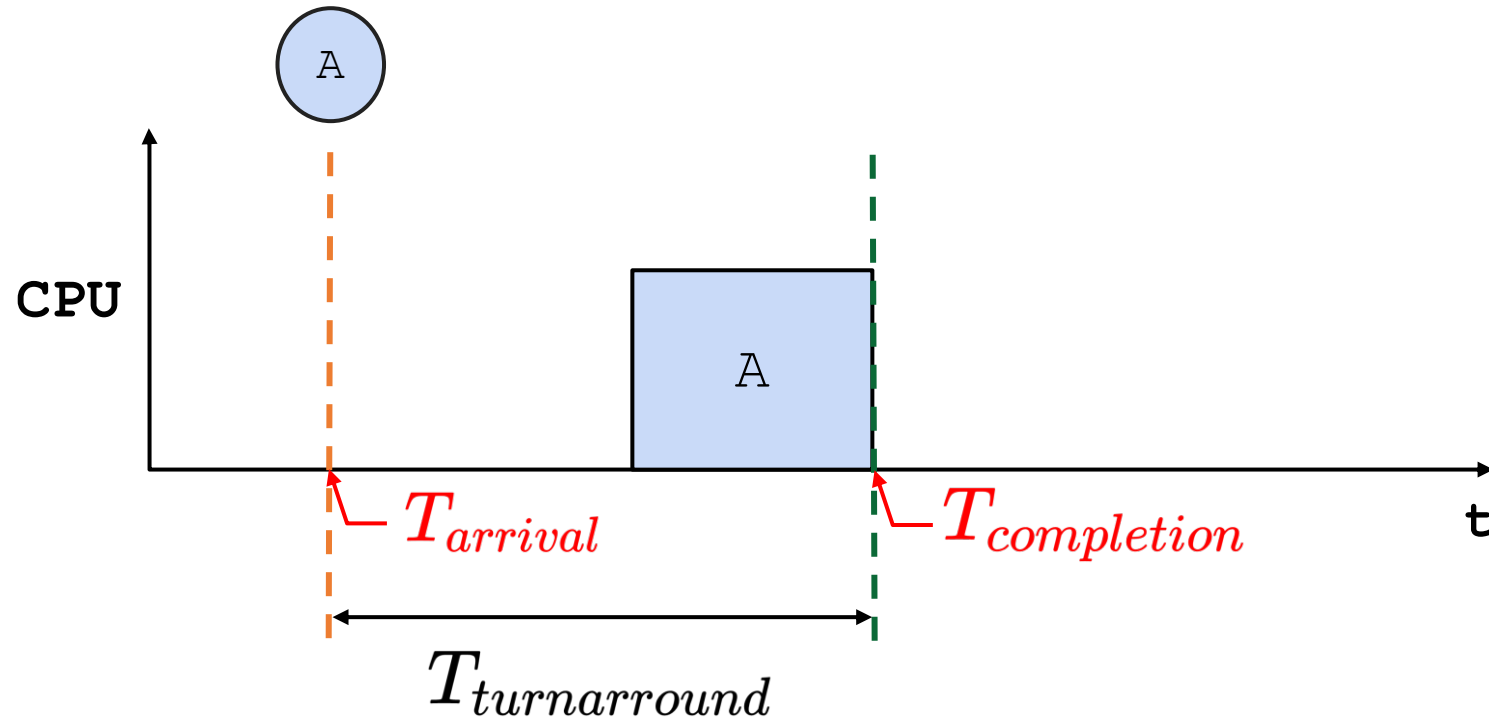
Repaso clase anterior

El Scheduler



Repaso clase anterior

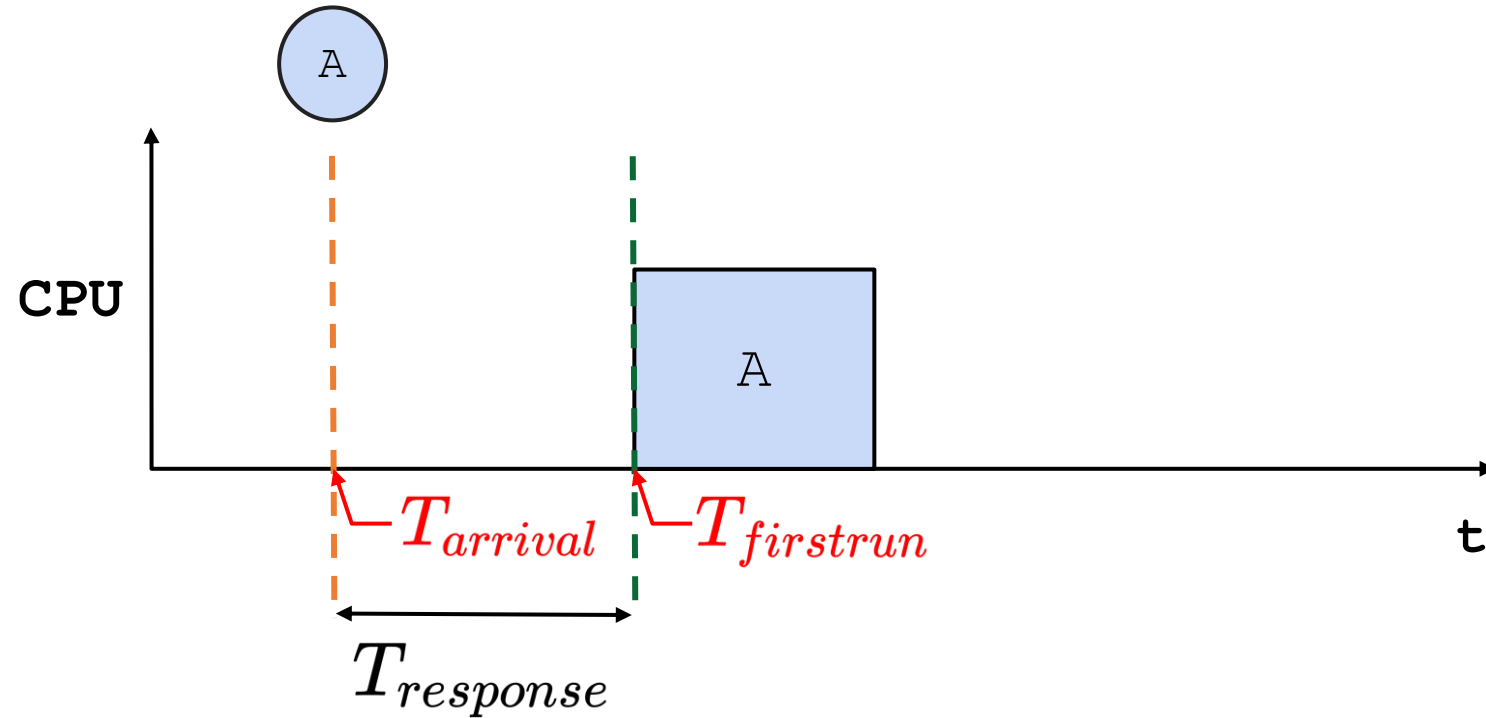
Métricas - Turnaround time



$$T_{turnaround} = T_{completion} - T_{arrival}$$

Repaso clase anterior

Métricas - Response Time



$$T_{response} = T_{firstrun} - T_{arrival}$$

Repaso clase anterior

Suposiciones ideales

1. Cada trabajo se ejecuta por la **misma cantidad de tiempo**
2. Todos los trabajos son **iniciados al mismo tiempo (arrival time)**
3. Una vez iniciado, cada trabajo **se ejecuta hasta su finalización**
4. Todos los trabajos solo **usan la CPU (no I/O)**
5. El **tiempo de ejecución** de cada trabajo es conocido (runtime).

A medida que avanzamos, las suposiciones se iban relajando para acercarnos más a la realidad



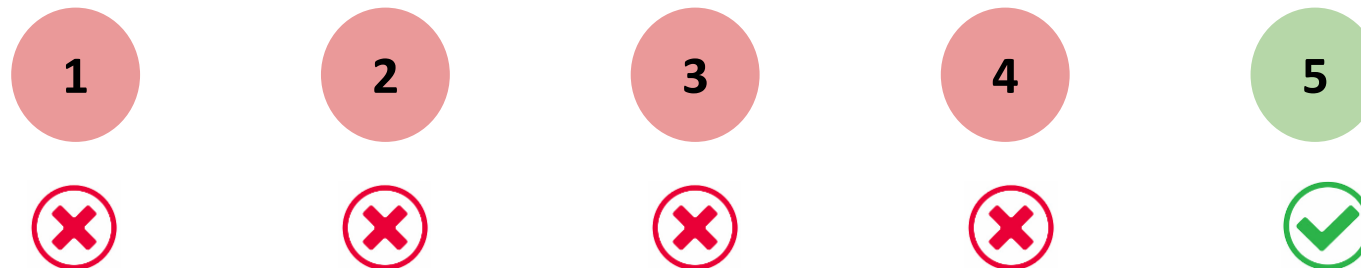
Repaso clase anterior

Suposiciones ideales

- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo~~
- ~~2. Todos los trabajos son iniciados al mismo tiempo (arrival time)~~
- ~~3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización~~
- ~~4. Todos los trabajos solo usan la CPU (no I/O)~~
5. El tiempo de ejecución de cada trabajo es conocido (runtime).

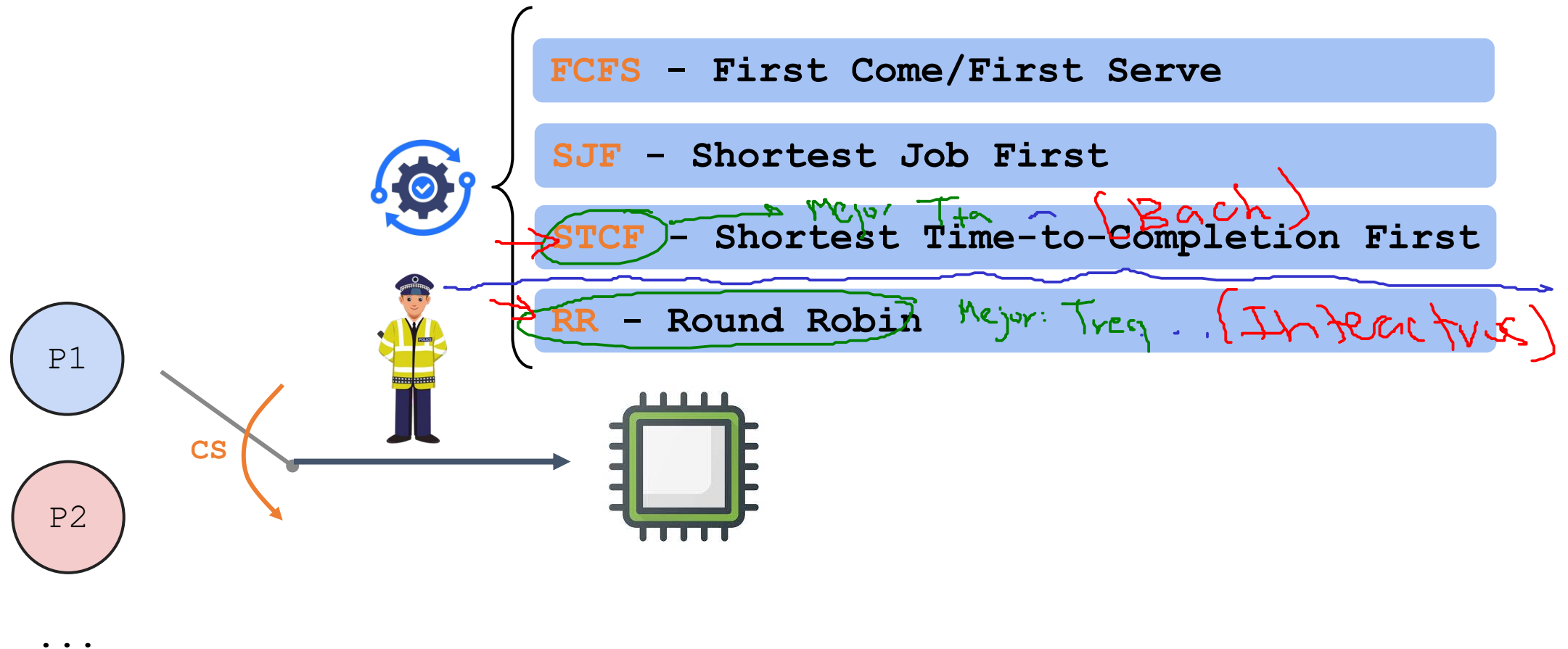
Como cuando va a vivir el proceso

¿Qué pasa si se relaja la última suposición ideal?



Repaso clase anterior

Algoritmos de planificación

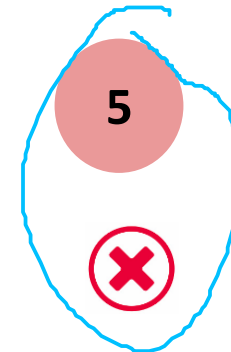
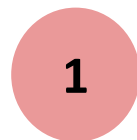


Contextualización

Observación: Afirmar que el tiempo de ejecución de cada proceso es conocido **es difícil**.

Suposiciones ideales

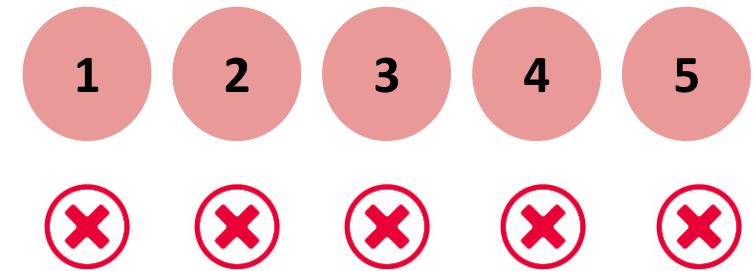
- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo~~
- ~~2. Todos los trabajos son iniciados al mismo tiempo (arrival time)~~
- ~~3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización~~
- ~~4. Todos los trabajos solo usan la CPU (no I/O)~~
- ~~5. El tiempo de ejecución de cada trabajo es conocido (runtime).~~



Contextualización

Punto de partida

- Al relajar todas las suposiciones ideales llegamos a la conclusión de que el Scheduler no conoce nada para tomar la decisión sobre cuál es el próximo proceso que ejecutará la CPU.

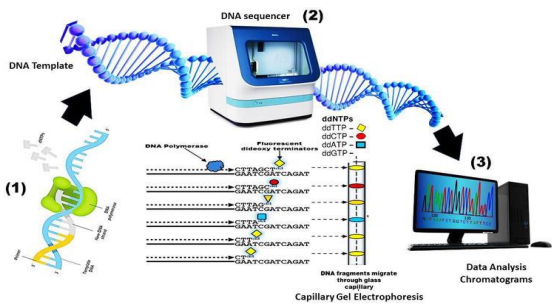
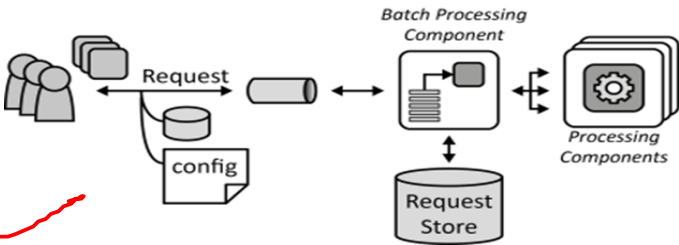
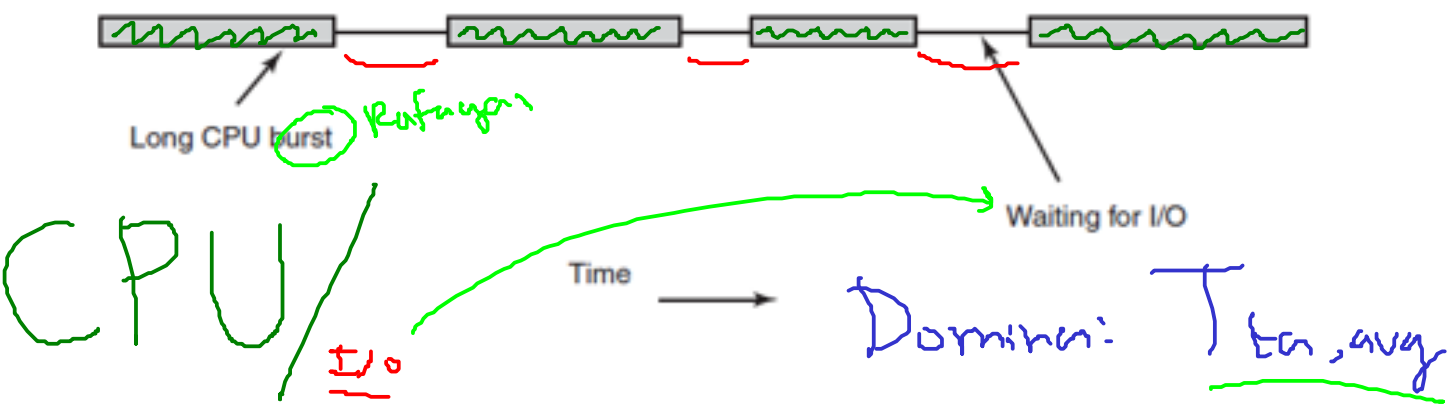


- ¿Cómo llevar a cabo el proceso de planificación sin tener un conocimiento perfecto de la situación?
- ¿Cómo diseñar un **planificador de propósito general** que funcione bien para todo tipo de procesos (interactivos y batch)?

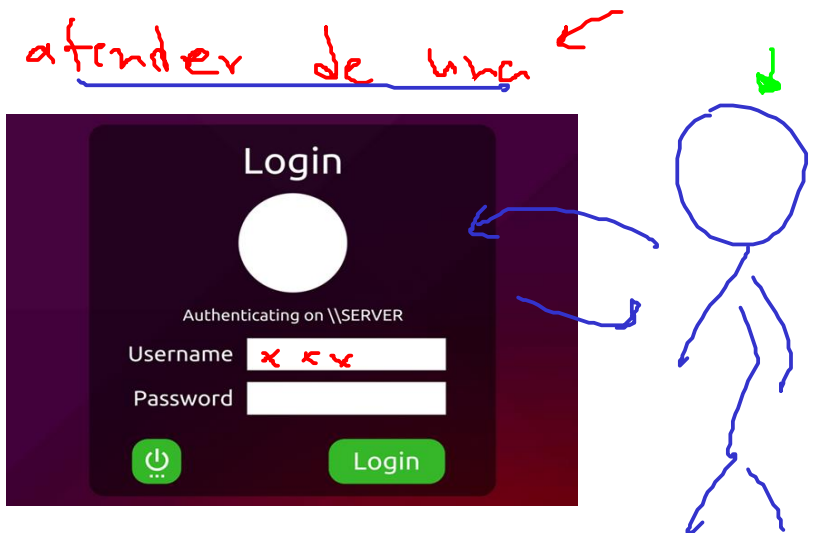
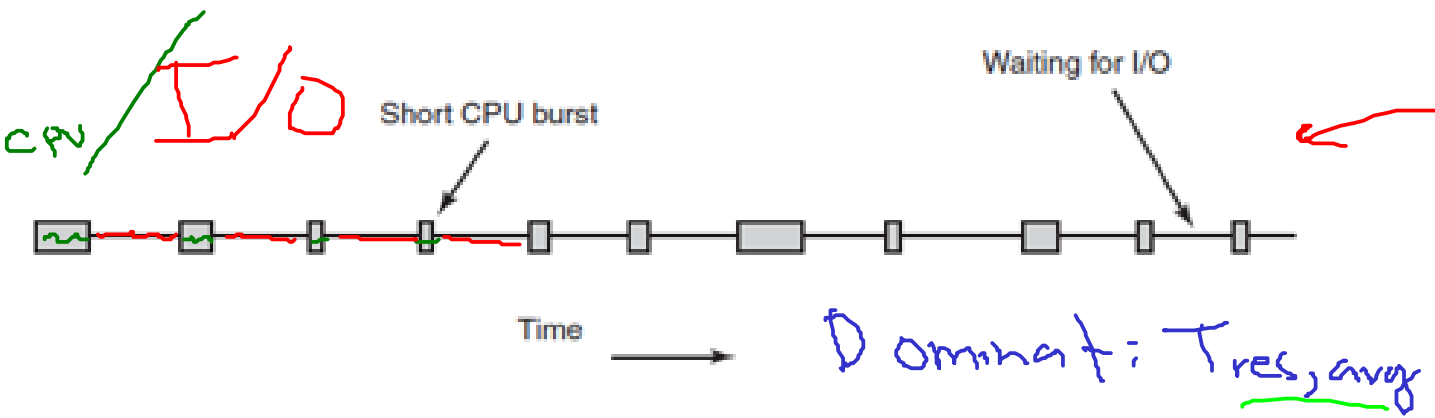
Tipos de procesos

Distinguimos dos tipos básicos de procesos:

1. Procesos tipo batch (CPU-bound process)



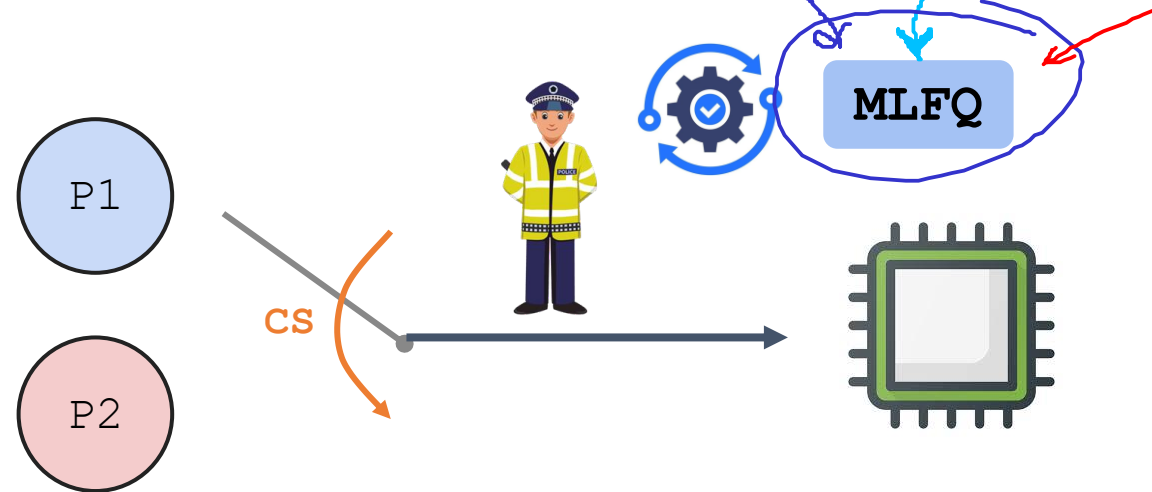
2. Procesos interactivos (I/O-bound process) $\rightarrow$ me hacen que



Planificador de propósito general

Batch
Interactivos.

- **Pregunta clave:** ¿Cómo diseñar un scheduler que aprenda de las características de los trabajos para tomar las mejores decisiones? (Aprender del pasado para predecir el futuro).
- **Objetivo:** Diseñar un planificador de propósito general capaz de soportar los dos tipos de procesos existentes:
 1. Interactivos (se preocupan por el tiempo de respuesta).
 2. Batch (se preocupan por el turnaround time).
- **Propuesta:** MLFQ (Multi-Level Feedback Queue)



MLFQ (Multi-Level Feedback Queue)

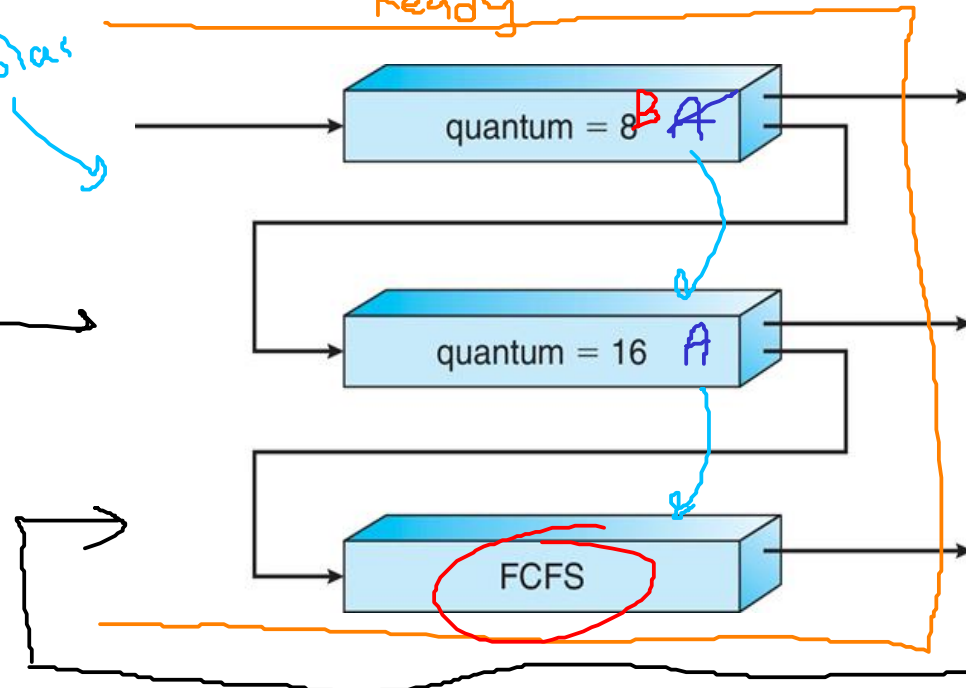
Conceptos importantes:

- Varias colas de procesos cada una con un nivel de prioridad diferente. ✓ *8 colas*
- Un proceso que este listo para su ejecución se ingresa en una de las colas.
- Cada proceso tiene una prioridad durante cierto tiempo pero esta puede cambiar.

~~B~~ A

3 colas

Ready



[High Priority]

Q8

A

B

Q7

Q6

Q5

Q4

C

Q3

Q2

[Low Priority]

Q1

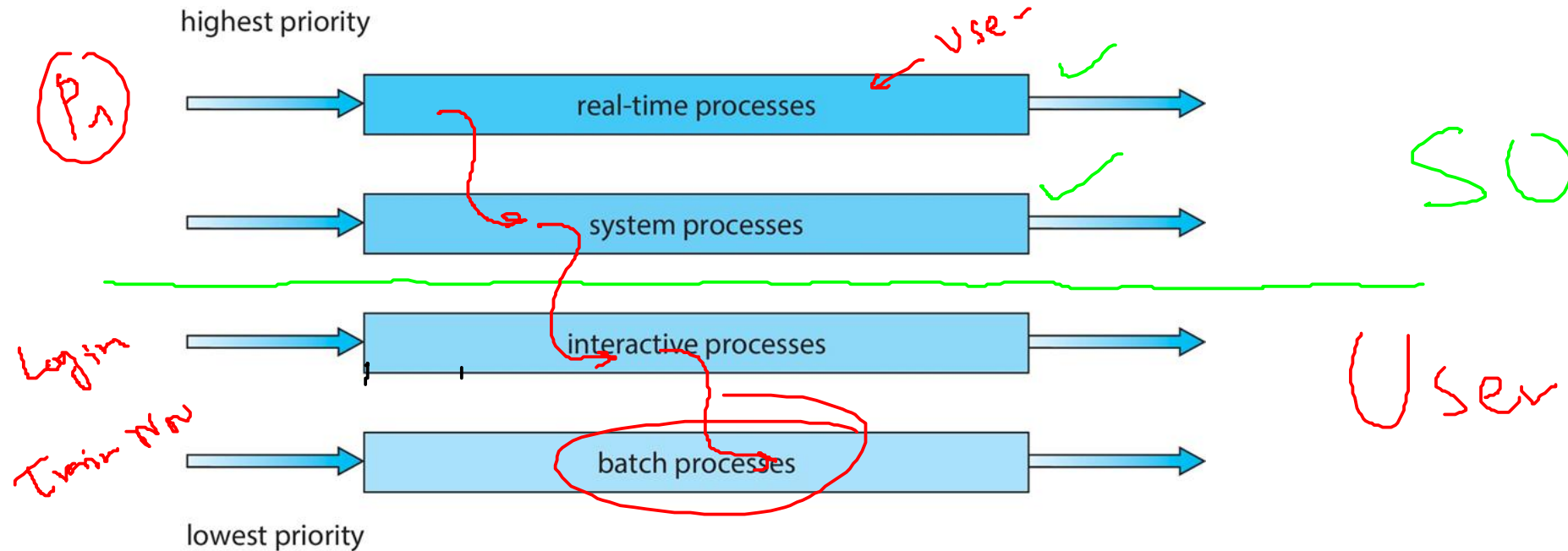
D

B)oc X

MLFQ (Multi-Level Feedback Queue)

La asignación de prioridades se realiza de acuerdo al comportamiento observado:

- **Alta prioridad:** Procesos I/O bound.
- **Bajas prioridades:** Procesos CPU-bound.



MLFQ - Reglas básicas

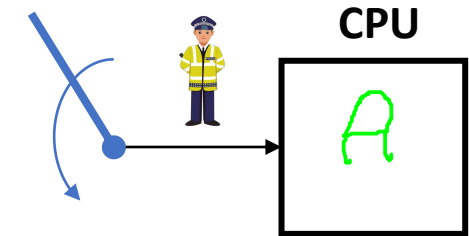
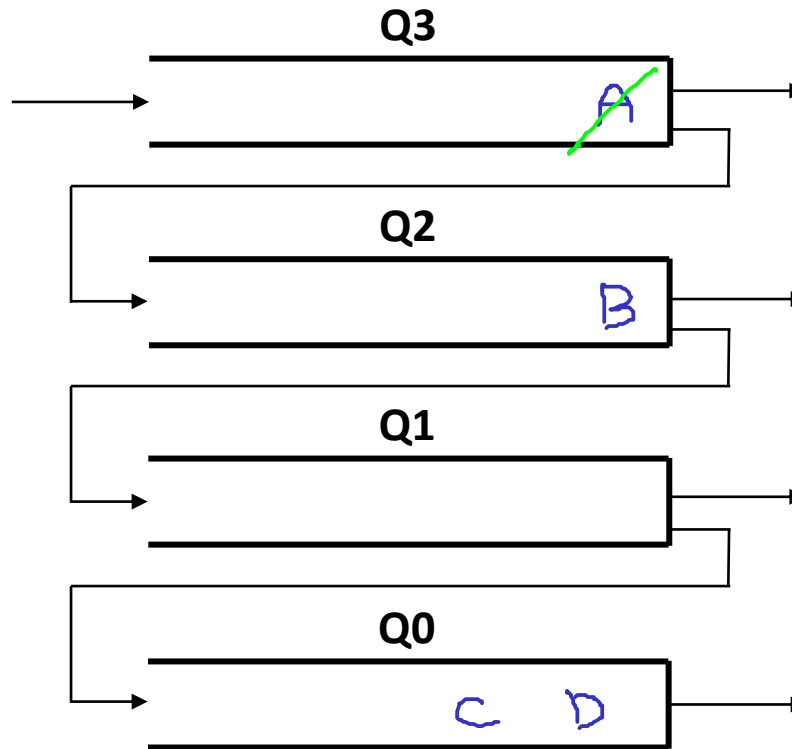
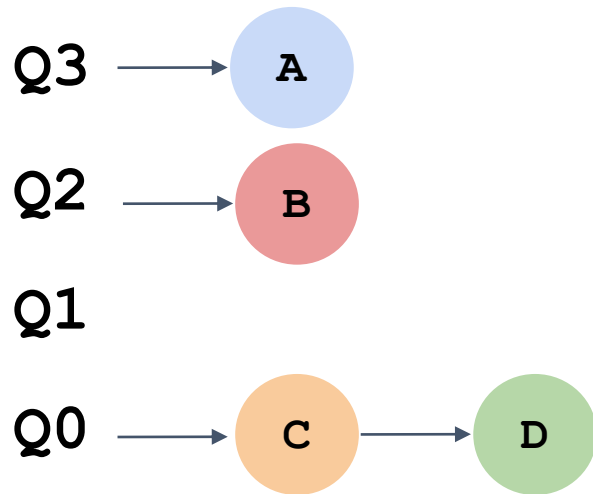
- **Regla 1:** Si **prioridad(A) > prioridad(B)**; A se ejecuta. ✓
- **Regla 2:** Si **prioridad(A) = prioridad(B)**; RR para A y B. ✓
- **Regla 3:** Cuando un trabajo llega al sistema es ubicado en la cola con la prioridad más alta. ✓
- **Regla 4a:** Si el trabajo usa completamente el quantum de tiempo, se reduce su prioridad. ✓
- **Regla 4b:** Si el trabajo entrega la CPU antes de finalizar su quantum de tiempo, mantiene el mismo nivel de prioridad.
- **Regla 5:** Después de un tiempo **S**, mueva todos los trabajos al mayor nivel de prioridad. ✓

A medida que avancemos en nuestro estudio iremos analizando la implicación de cada una de estas reglas.

MLFQ - Reglas básicas

Prioridades

- Regla 1: Si **prioridad(A) > prioridad(B)**; A se ejecuta.
- Regla 2: Si **prioridad(A) = prioridad(B)**; RR para A y B.



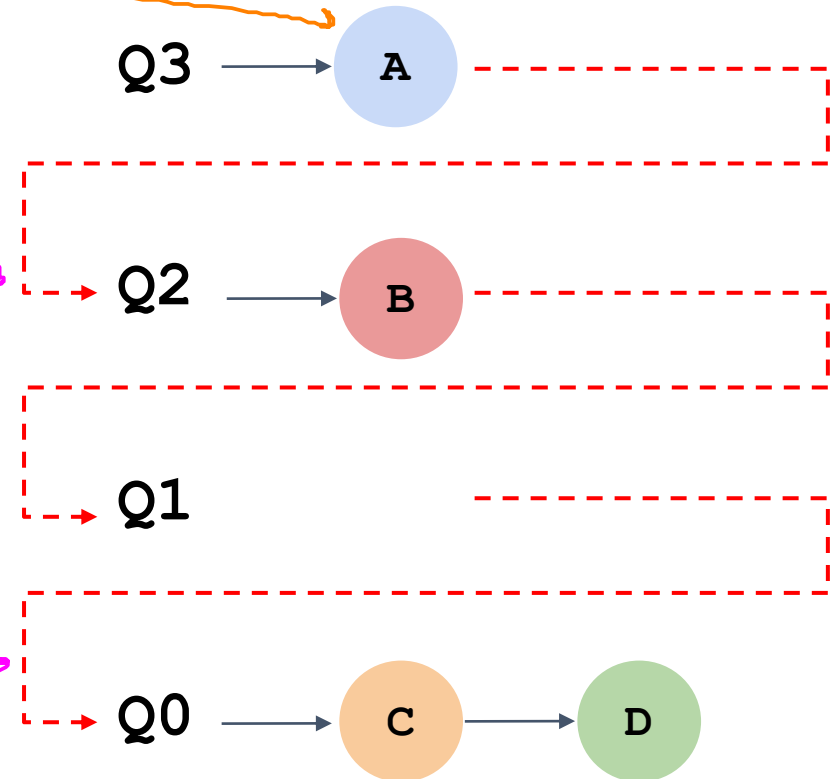
MLFQ - Reglas básicas

Realidad: El **workload** tiene una mezcla de trabajos interactivos y tipo batch.

Cambio de prioridad

- **Regla 3:** Cuando un trabajo llega al sistema es ubicado en la cola con la prioridad más alta.
- **Regla 4a:** Si el trabajo usa completamente el quantum de tiempo, se reduce su prioridad.
- **Regla 4b:** Si el trabajo entrega la CPU antes de finalizar su quantum de tiempo, mantiene el mismo nivel de prioridad.

Nuevos



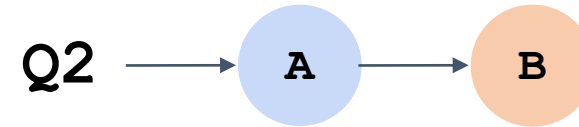
Si el proceso se queda en la misma cola o se mueve a otra de menor prioridad.

MLFQ – Proceso CPU-bound

q₂ (va despues de la que sigue)

Ejemplo 3 - Proceso I/O bound

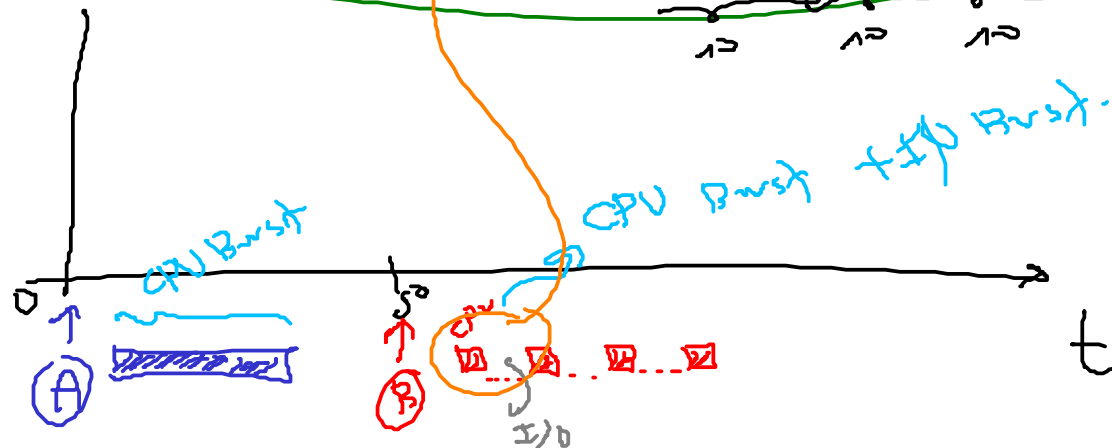
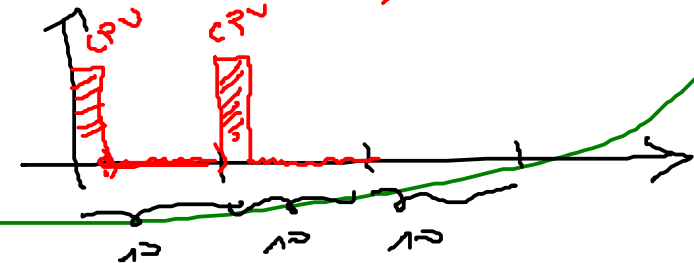
- MLFQ de 3 colas (Q2, Q1, Q0).
- Quantum de 10 ms para cada cola.
- Procesos:
 - **A:**
 - Proceso con uso intensivo de la CPU (CPU-bound).
 - **B:**
 - Proceso I/O bound.
 - Usa la CPU 1 ms y realiza una operación I/O.
 - Llega en t = 50 ms.



Q1

Q0

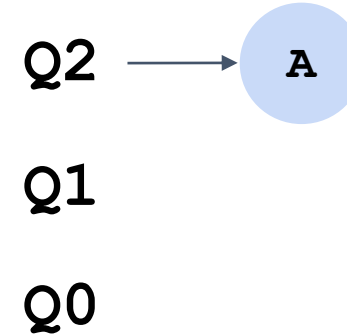
CPU + I/O
1/10 9/10



MLFQ – Proceso CPU-bound

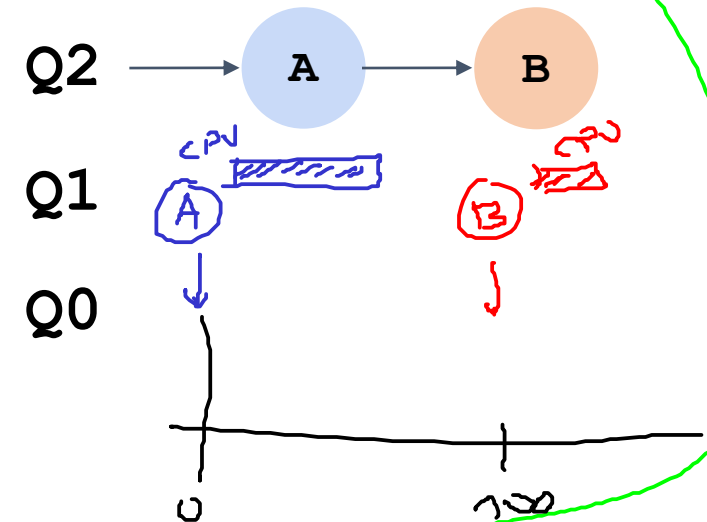
Ejemplo 1 - Proceso CPU bound

- MLFQ de 3 colas (Q2, Q1, Q0). ✓
- Quantum de 10 ms para cada cola.
- Solo un proceso: A.



Ejemplo 2 - Llega un proceso corto

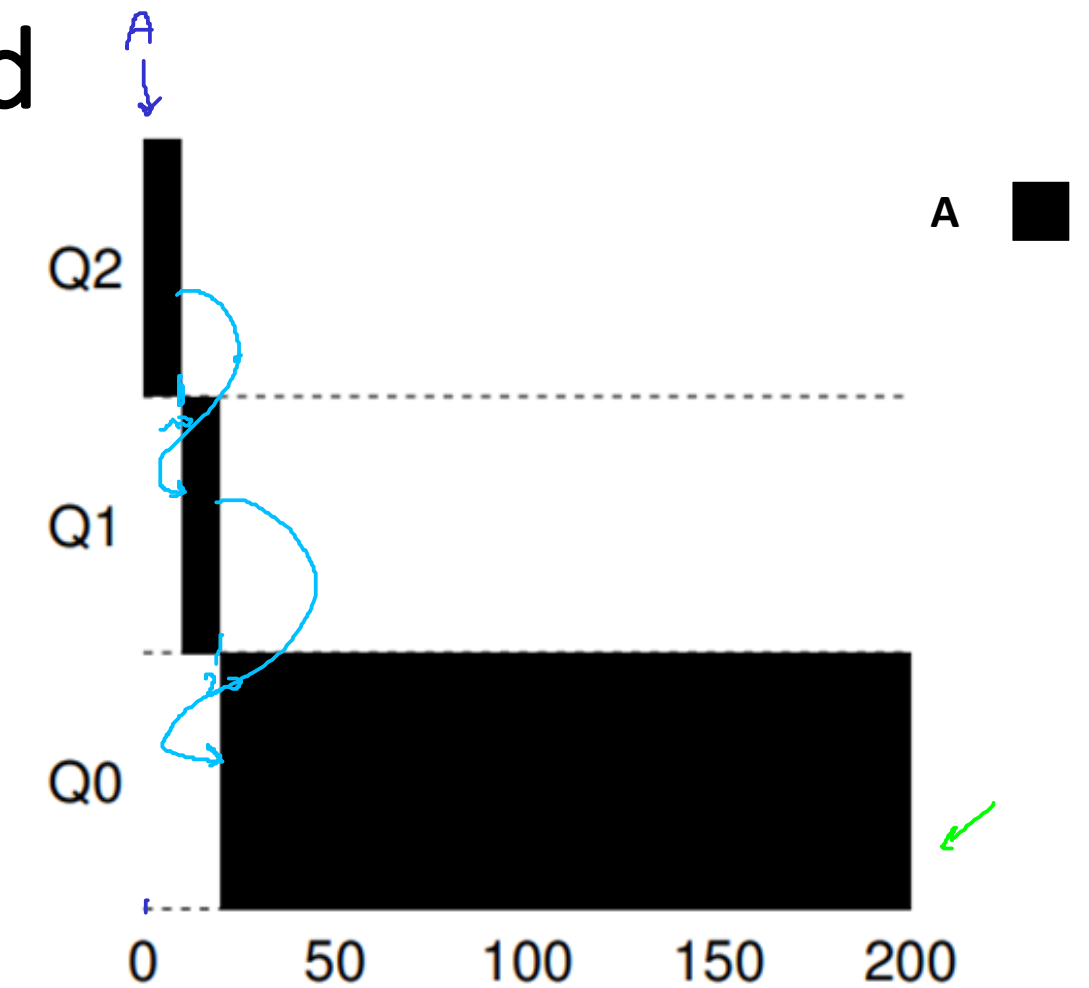
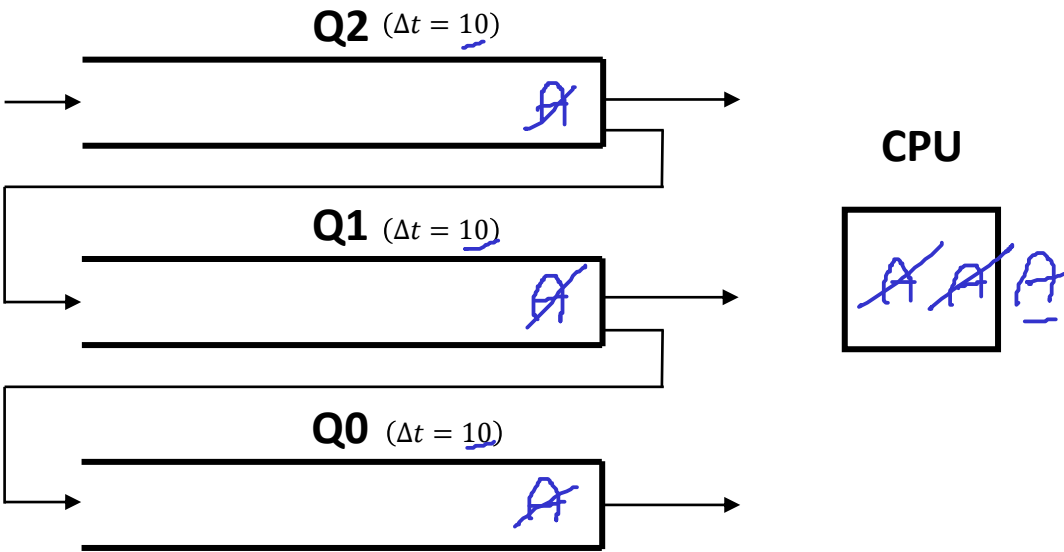
- MLFQ de 3 colas (Q2, Q1, Q0).
- Quantum de 10 ms para cada cola.
- Procesos:
 - A:
 - Proceso con uso intensivo de la CPU (CPU-bound).
 - B:
 - Proceso ~~interactivo~~ CPU (10, 20).
 - Tiempo de ejecución de 20 ms.
 - Llega en $t = 100$.



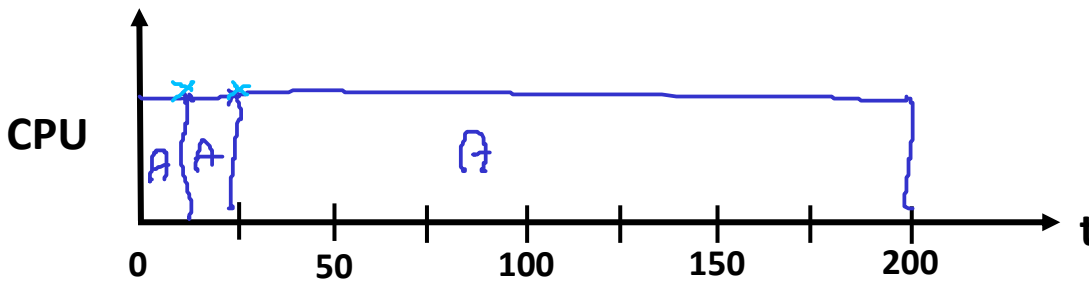
MLFQ – Proceso CPU-bound

Ejemplo 1 - Proceso CPU bound

Proceso	Arrival time	Run-time
<u>A</u>	0	?? 200+ 0 ms



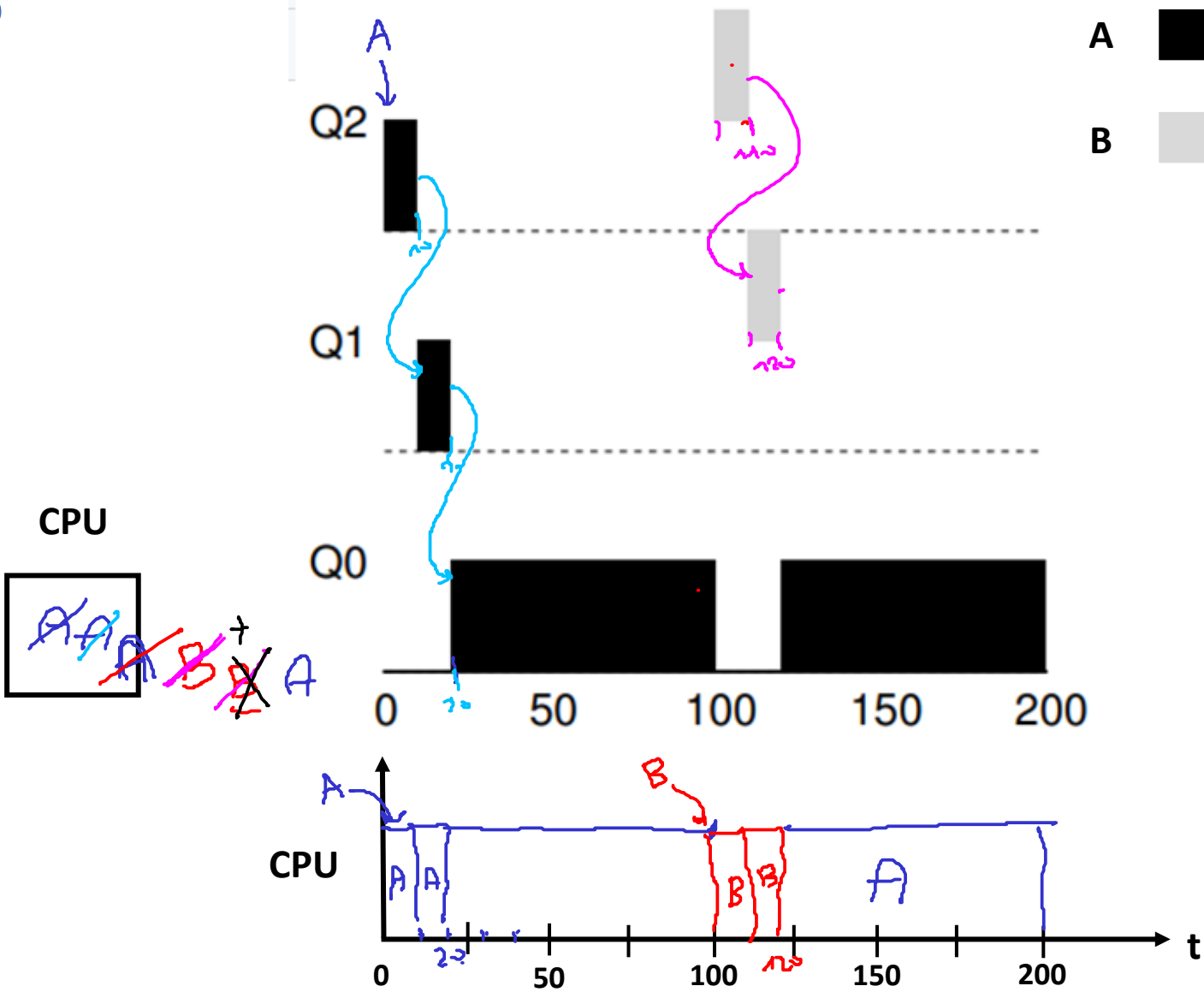
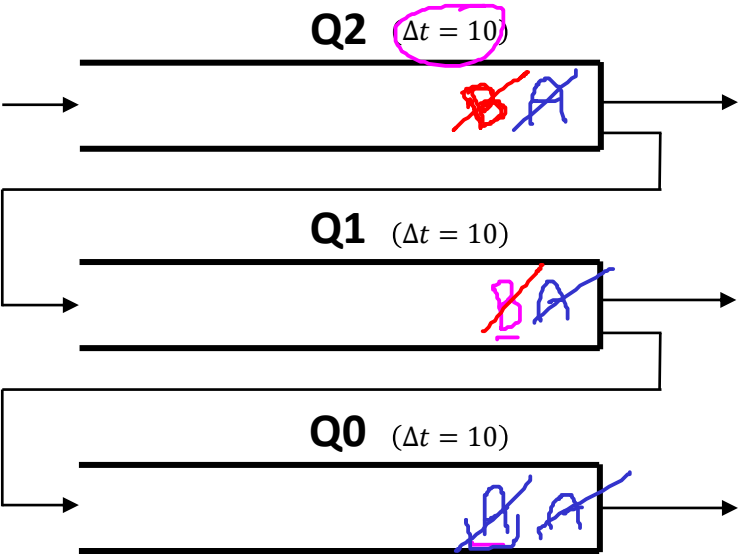
Análisis: El proceso (Regla 3) empieza con la más alta prioridad y (Regla 4) migra a una prioridad más baja cuando usa su quantum de tiempo en la cola en la que está (Reglas 3 y 4a)



MLFQ (Multi-Level Feedback Queue)

Ejemplo 2 - Llega un proceso corto

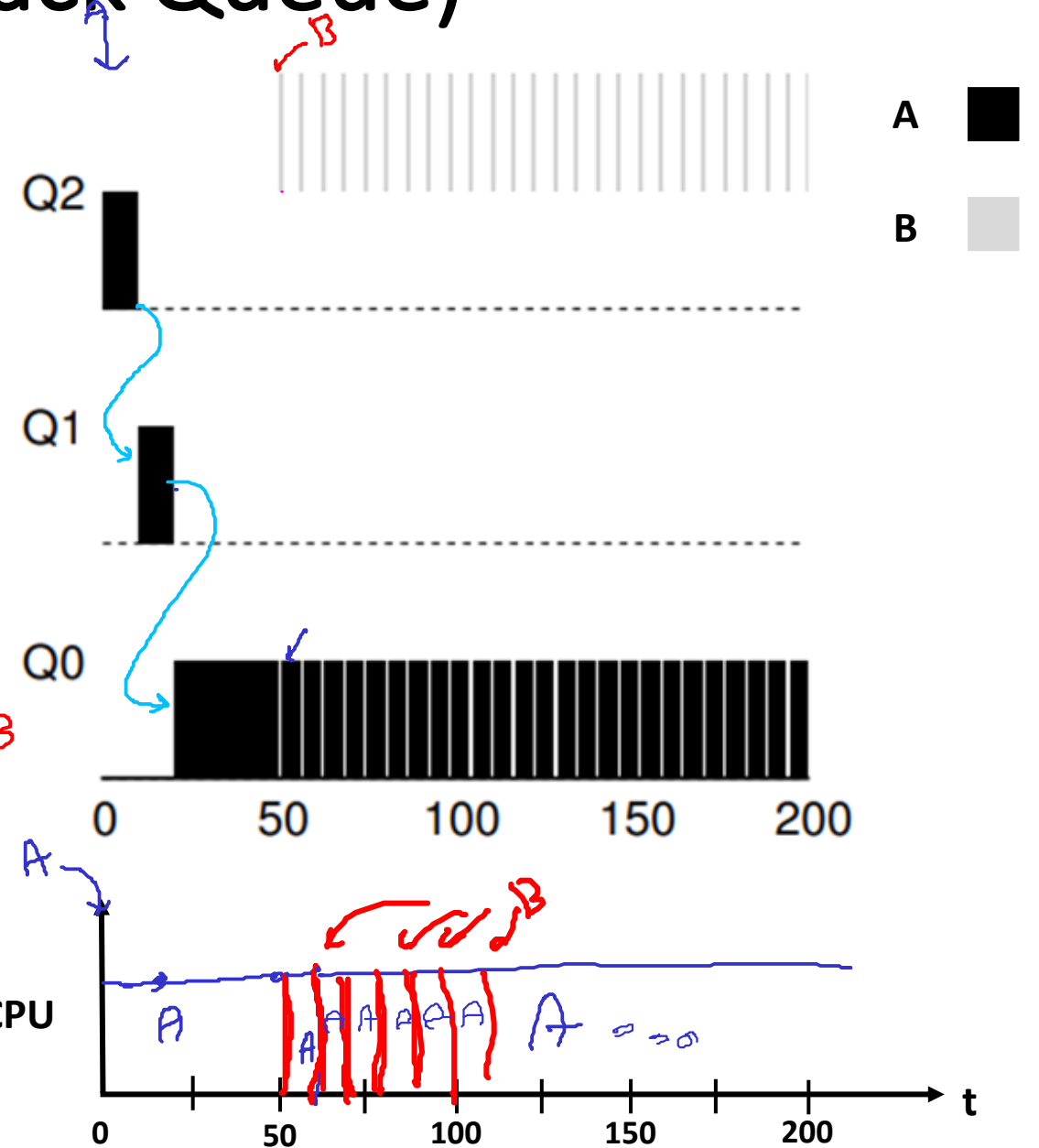
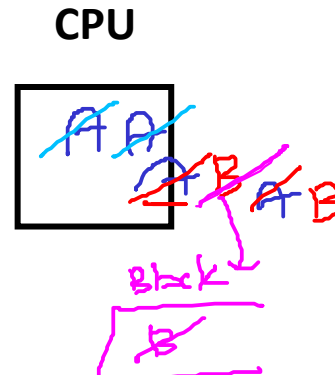
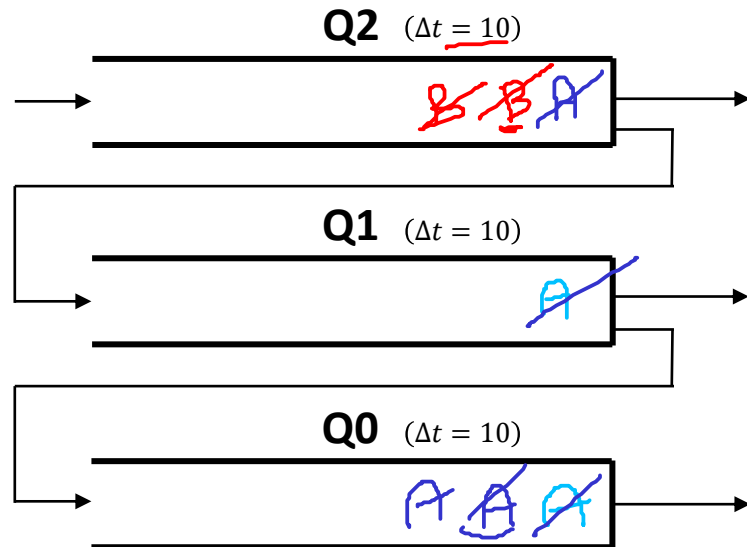
Proceso	Arrival time	Run-time
A	0 ✓	200 +
B	100	20



MLFQ (Multi-Level Feedback Queue)

Ejemplo 3 - Proceso I/O bound

Proceso	Arrival time	Run-time
A	0	...
<u>B</u>	50	- CPU-Burst: 1 ms - I/O-Burst: 9m

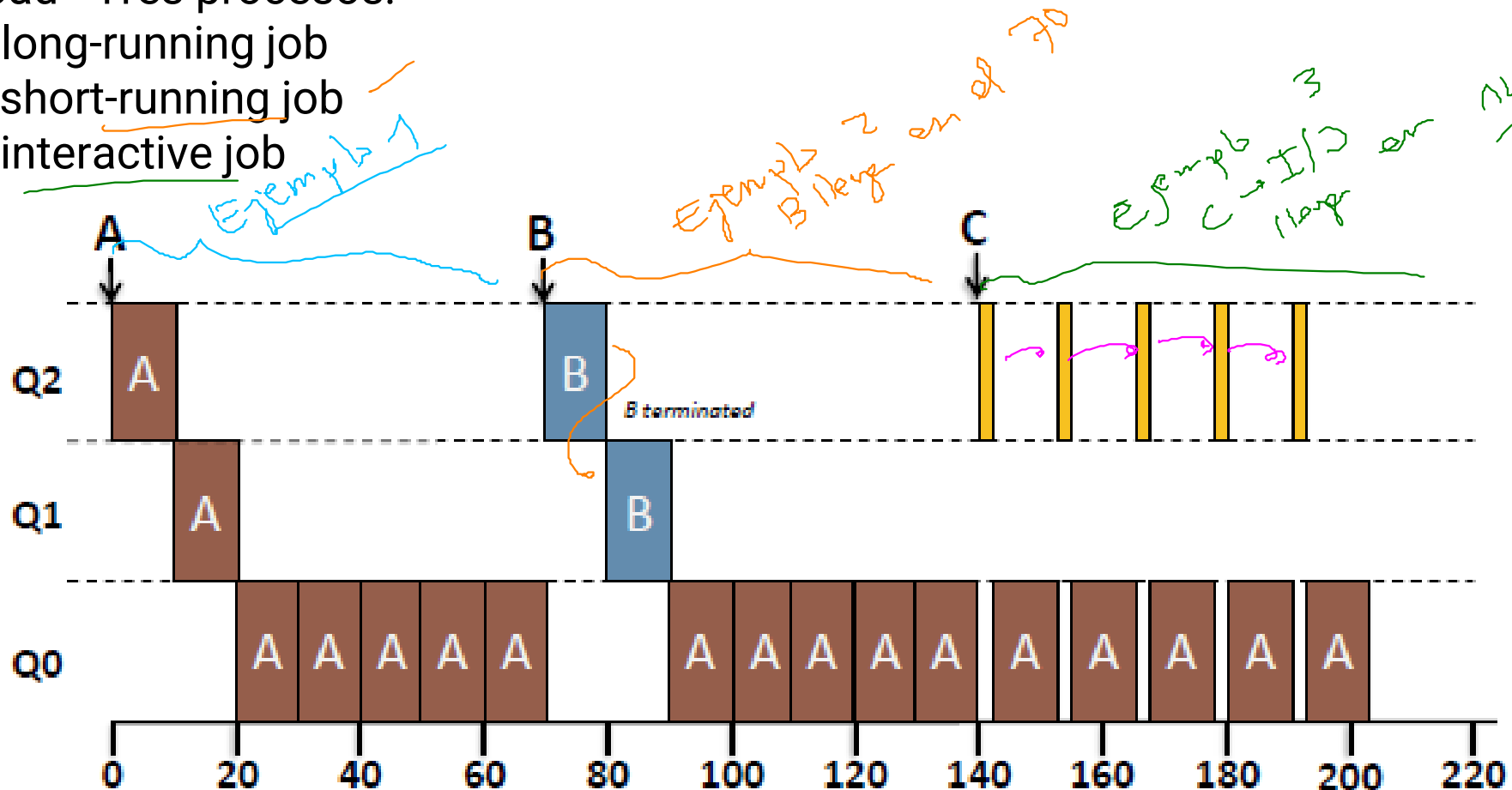


MLFQ mantiene al proceso interactivo con la prioridad más alta.

MLFQ – Proceso CPU-bound

Ejemplo resumen

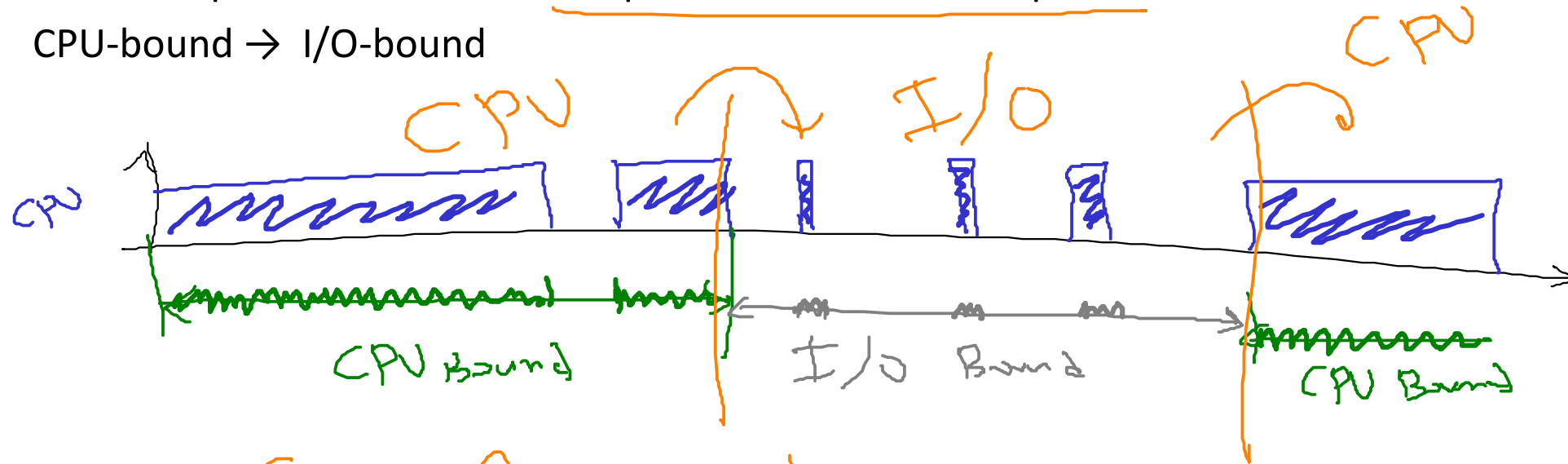
- MLFQ de 3 colas (Q2, Q1, Q0) de quantum 10ms.
- Workload - Tres procesos:
 - **A**: long-running job
 - **B**: short-running job
 - **C**: interactive job



Problemas del MLFQ básico - Cambio de comportamiento

Problema: Un proceso cambia su comportamiento en el tiempo

- CPU-bound $\rightarrow$ I/O-bound



Como funciona lo que hemos visto
hasta el momento para este caso?

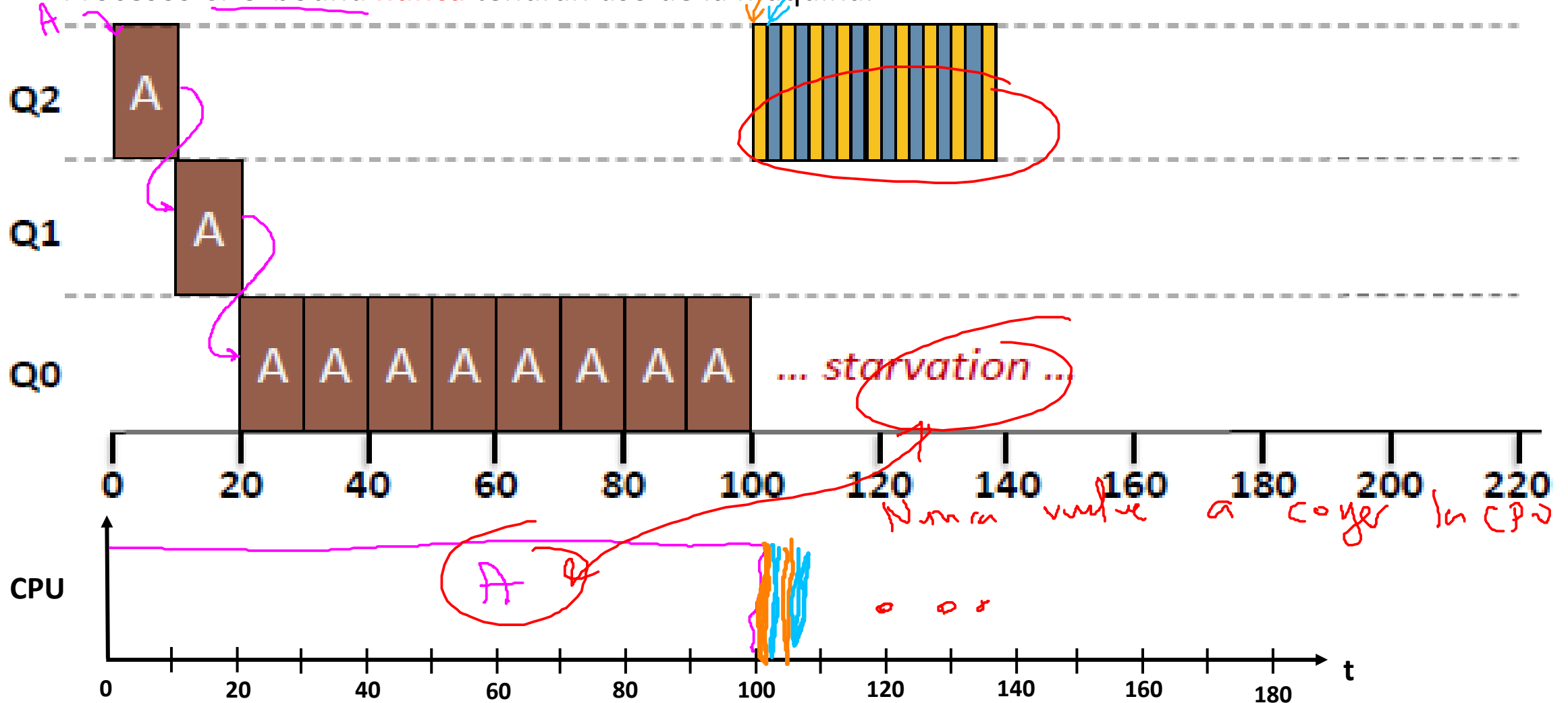
Problema: { ① Inanidad

$\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4a, 4b \end{pmatrix}$

Problemas del MLFQ básico – Inanición

Problema: Inanición (starvation)

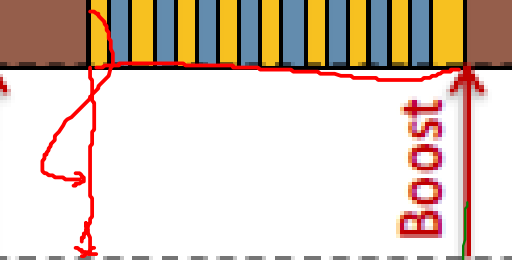
- Muchos procesos interactivos.
- Procesos CPU-bound **nunca** tendrán uso de la máquina.



$$\begin{pmatrix} 1. \\ 2. \\ 3. \\ 4a, 4b \\ 5 \end{pmatrix}$$

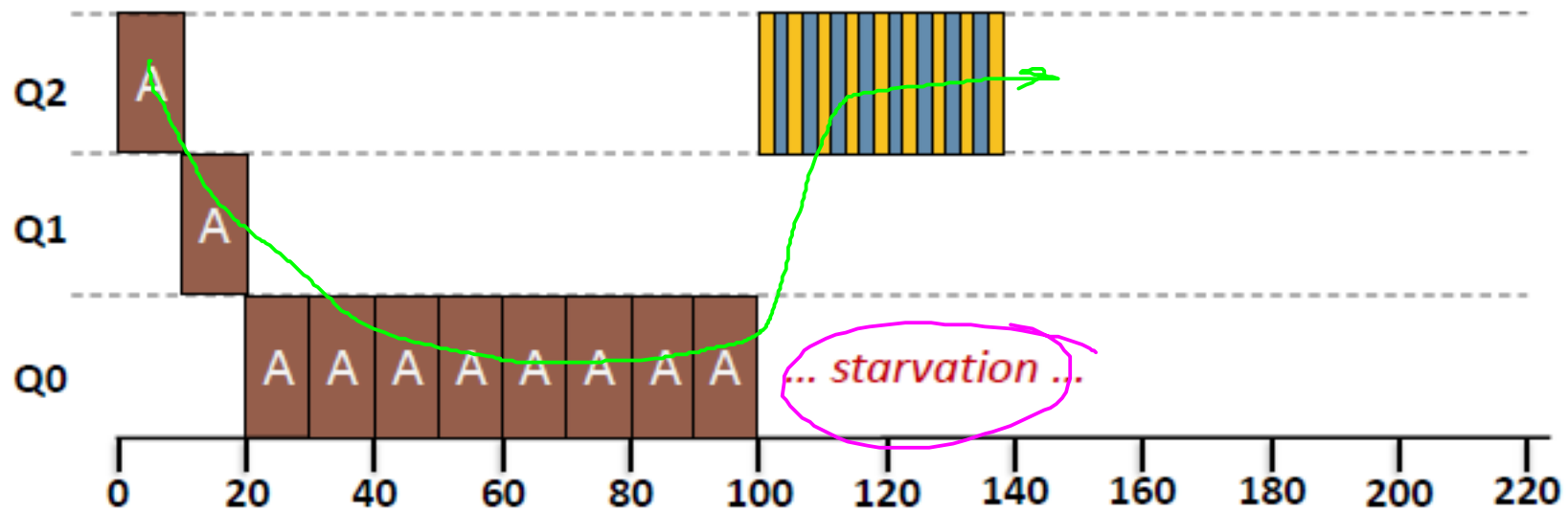
- News -

Después de un tiempo S, mueva to

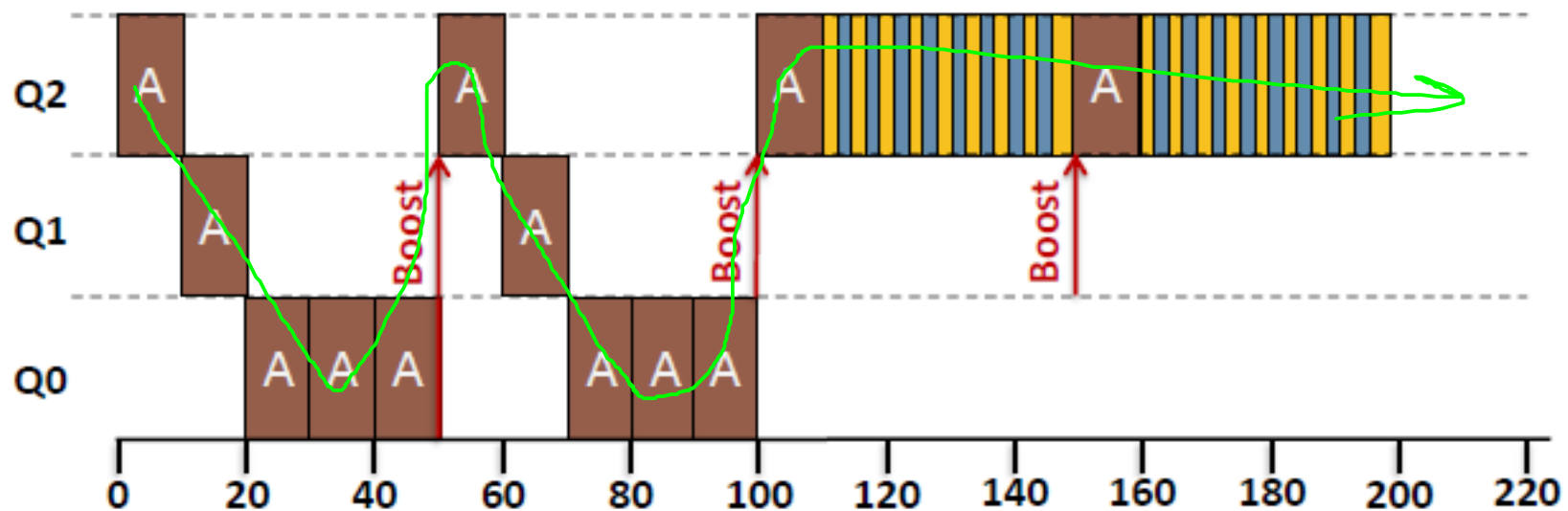


Priority boost

Without
Priority
Boost



With
Priority
Boost



MLFQ - Clásico

Reglas básicas

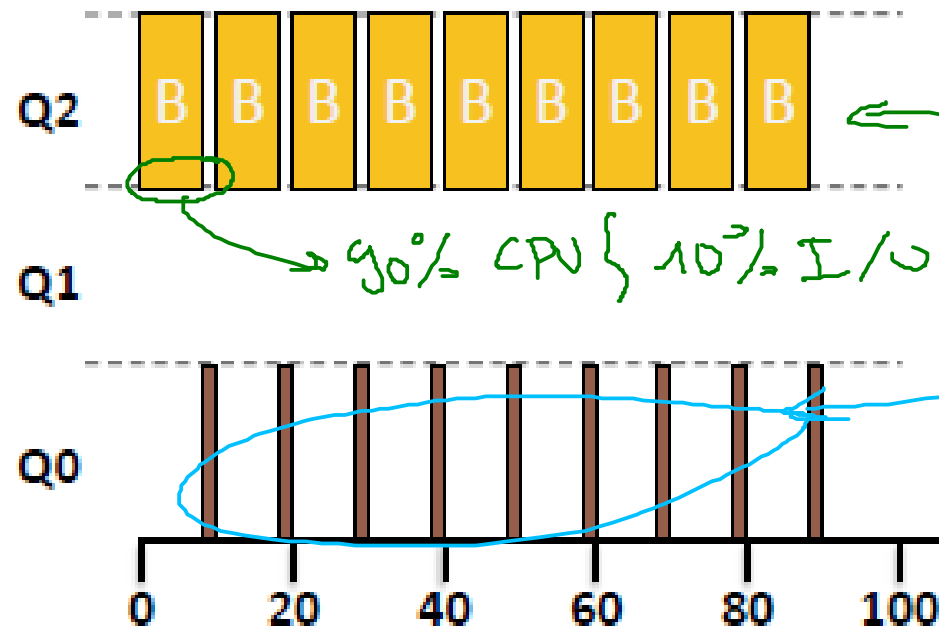
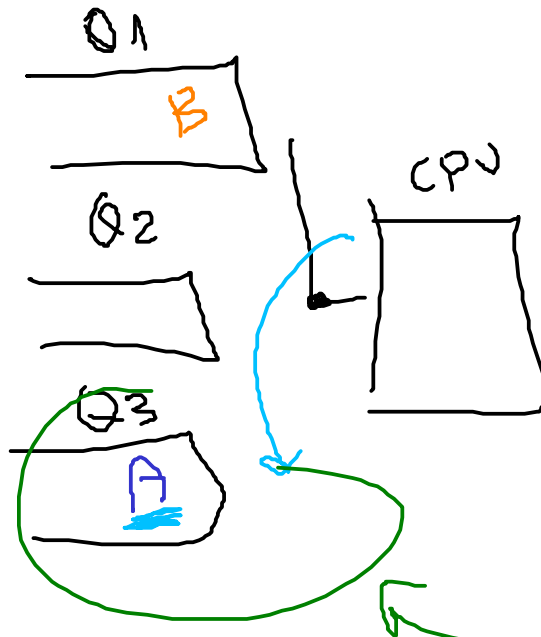
- **Regla 1:** Si **prioridad(A) > prioridad(B)**; A se ejecuta. ✓
- **Regla 2:** Si **prioridad(A) = prioridad(B)**; RR para A y B.
- **Regla 3:** Cuando un trabajo llega al sistema es ubicado en la cola con la prioridad más alta. ✓
- **Regla 4a:** Si el trabajo usa completamente el quantum de tiempo, se reduce su prioridad. ✓
- **Regla 4b:** Si el trabajo entrega la CPU antes de finalizar su quantum de tiempo, mantiene el mismo nivel de prioridad.
- **Regla 5:** Después de un tiempo S, mueva todos los trabajos al mayor nivel de prioridad. ✓

Problemas del MLFQ básico - Engañar al planificador

Problema: Engañar al planificador (Game the scheduler).

Hecho la ley hecho la trampa

- Un proceso malicioso puede engañar al planificador liberando la CPU justo antes de que expire el quantum (intervalo de tiempo).
 - Proceso se ejecuta el 99% del quantum y luego realiza una operación I/O.
 - Obtiene mayor uso de la máquina.



modif.

Se como la CPU

la obtiene poco

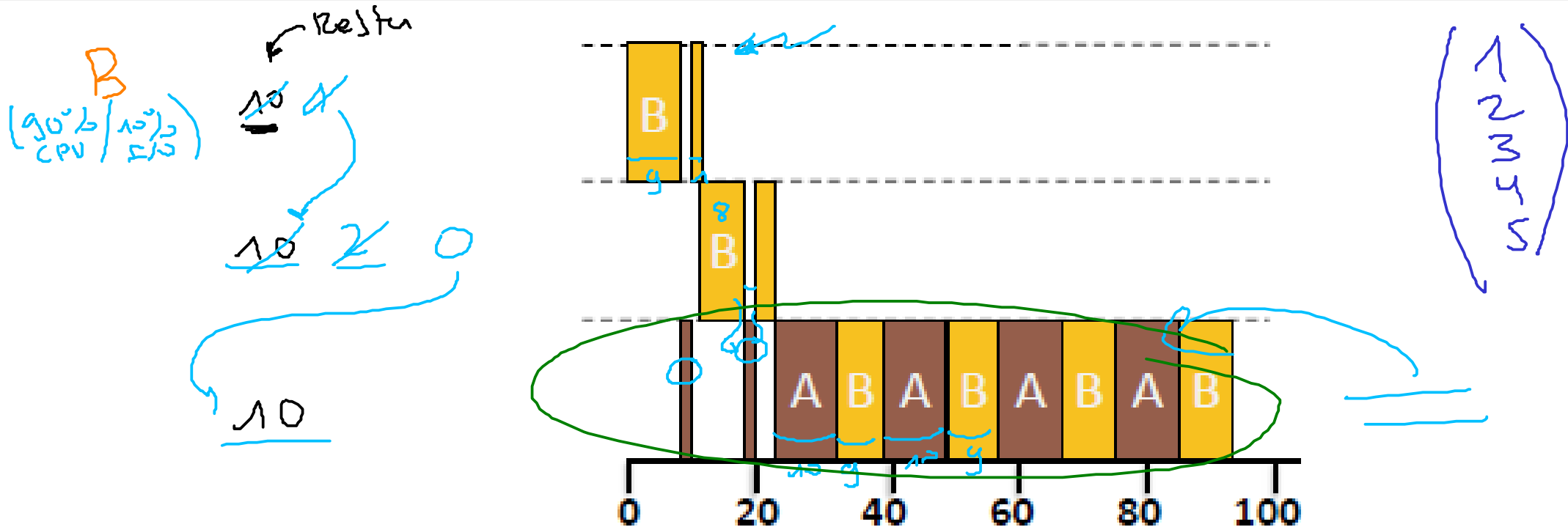
Necesitamos una enmendona

Without precise accounting

Problemas del MLFQ básico - Engañar al planificador

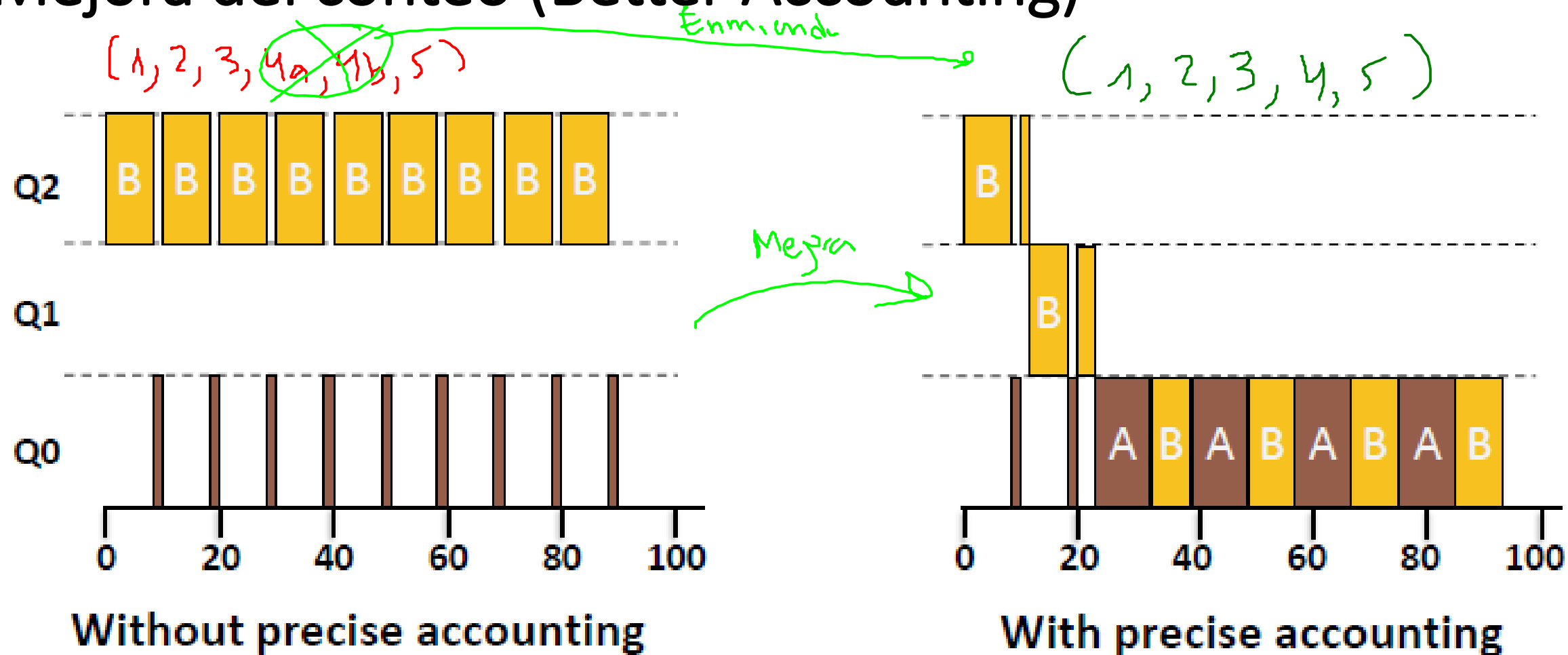
- Solución: Mejora del conteo

Nueva regla 4 (reemplaza 4a y 4b): Cuando un trabajo consume su asignación de tiempo en un nivel su prioridad es reducida (sin importar las veces que este retome el uso de la CPU).



With precise accounting

Mejora del conteo (Better Accounting)



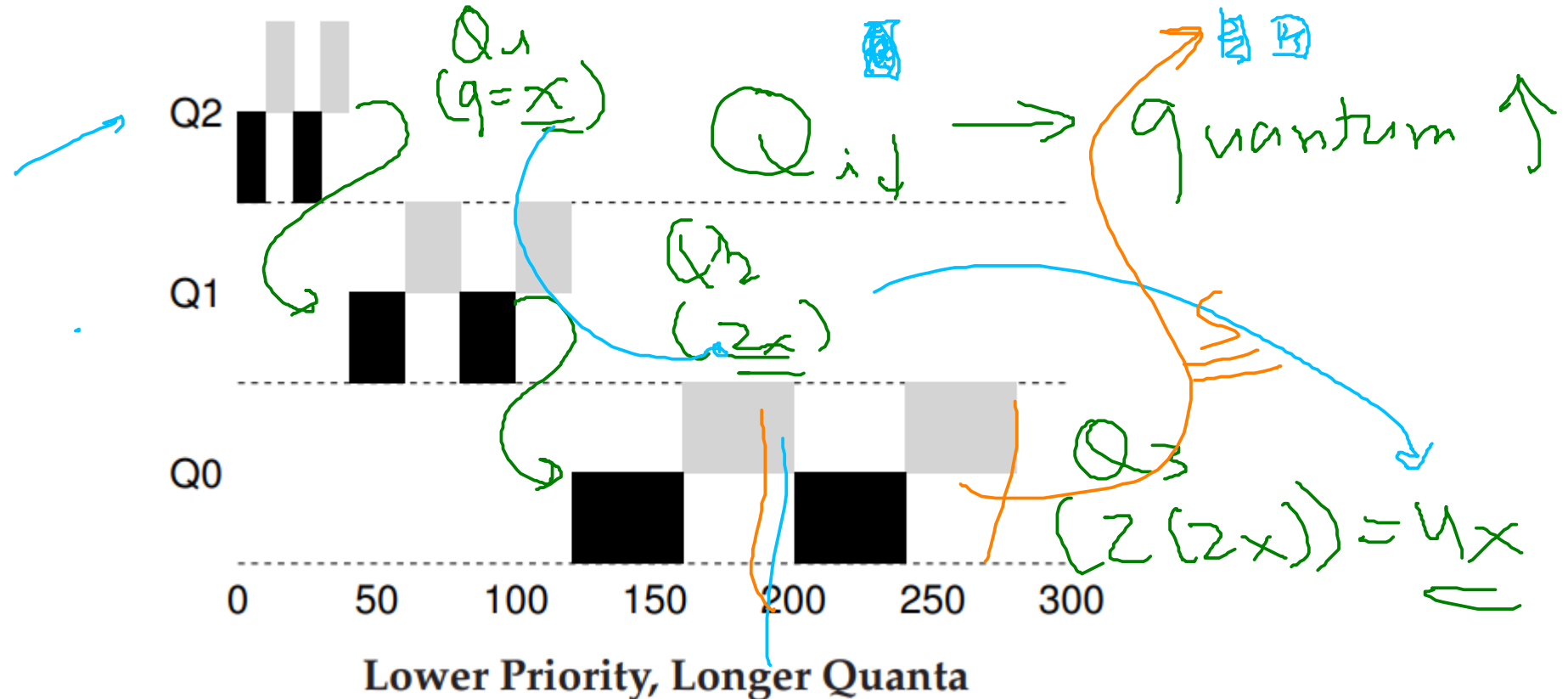
MLFQ - Mejorado

Reglas básicas

- **Regla 1:** Si **prioridad(A) > prioridad(B)**; A se ejecuta. ✓
- **Regla 2:** Si **prioridad(A) = prioridad(B)**; RR para A y B. ✓
- **Regla 3:** Cuando un trabajo llega al sistema es ubicado en la cola con la prioridad más alta.
- **Nueva Regla 4:** Cuando un trabajo consume su asignación de tiempo en un nivel su prioridad es reducida (sin importar las veces que este retome el uso de la CPU).
- **Regla 5:** Después de un tiempo **S**, mueva todos los trabajos al mayor nivel de prioridad

Mejoras al MLFQ

- Menor nivel de prioridad → mayor tiempo de quantum
 - **Alta prioridad:** quantum corto (~ 10 ms)
 - **Baja prioridad:** quantum largo (~ 100 ms)



Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <https://github.com/lis1-sys-prog-course/docs>
- <https://www.merlot.org/merlot/viewMaterial.htm?id=773407108>
- <https://github.com/silentzero192/Operating-Systems-Lab/tree/main>
- <https://github.com/getvmio/free-operating-system-resources>